



Universidad Católica Boliviana "San Pablo"
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

La Paz - Bolivia



Desarrollo de un Sistema de
Información Organizacional Web
basado en el enfoque de
Procesamiento de Transacciones
(TPS) para la gestión de procesos,
usuarios, transacciones y reportes
para una cafetería

Integrantes:

- Claros Suntura Juan Jose (*Desarrollador FullStack*)
- Lecoña Condori Elias Milan (*Desarrollador FullStack*)
- Maldonado Carvajal Alan Ariel (*Scrum Master*)
- Torrez Calle Alvaro Ariel (*Product Owner*)

Materia: Ingeniería de Software (SIS-213)

Docente: Miguel Angel Pacheco Arteaga

Fecha: 10/04/2026

Universidad: Universidad Católica Boliviana

Índice General

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS	1
1.1 Introducción	1
1.2 Antecedentes	2
1.2.1 Antecedentes del objeto de estudio	2
1.2.2 Referencias técnicas de otros sistemas TPS	3
1.3 Descripción del objeto de estudio	5
1.4 Descripción de Procesos del Sistema	7
1.4.1 Proceso 1: Autenticación y Control de Acceso	7
1.4.2 Proceso 2: Gestión del Catálogo (Administrador)	7
1.4.3 Proceso 3: Registro de Orden en el POS (Cajero)	8
1.4.4 Proceso 4: Procesamiento Transaccional y Cierre (Backend)	8
1.4.5 Proceso 5: Generación de Comprobante (Factura/Ticket)	8
1.4.6 Proceso 6: Reportes y Arqueo de Caja	8
1.5 Formulación del Problema	9
1.6 Objetivos	9
1.6.1 Objetivo General	9
1.6.2 Objetivos Específicos	9
1.7 Análisis preliminar del sistema TPS	11
1.8 Propuesta de solución	12
1.9 Cronograma	13
Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS	16
2.1 Sistemas de Información Organizacional	16
2.1.1 Definición	16
2.1.2 Componentes	16
2.1.3 Tipos de sistemas	17
2.2 Sistema de Procesamiento de Transacciones (TPS)	17
2.2.1 Definición	17

2.2.2	Características	18
2.2.3	Funciones	19
2.2.4	Evolución hacia sistemas web	19
2.3	Arquitectura de sistemas web	20
2.3.1	Cliente	20
2.3.2	Servidor	20
2.3.3	API	21
2.3.4	Base de datos	21
2.3.5	Flujo del Sistema	21
2.4	Seguridad en sistemas de información	21
2.4.1	Autenticación	21
2.4.2	Autorización	22
2.4.3	Roles	22
2.4.4	Control de acceso	22
2.5	Base de datos	22
2.5.1	Modelo relacional	23
2.5.2	Integridad	23
2.5.3	Normalización	23
2.5.4	Transacciones	23
2.6	Metodología de desarrollo	24
2.6.1	Scrum	24
Capítulo 3. MARCO PRÁCTICO — DESARROLLO DEL SISTEMA TPS		26
3.1	Análisis del sistema	26
3.2	Determinación de requerimientos	27
3.2.1	Requerimientos funcionales	27
3.2.2	Requerimientos no funcionales	27
3.3	Modelado del sistema	28
3.3.1	Historias de Usuario	28
3.3.2	Diagramas UML	28

3.4	3.4 Diseño del sistema	29
3.4.1	Arquitectura del sistema — Modelo C4	29
3.5	3.5 Diseño de la Base de Datos	33
3.5.1	Paradigma de persistencia	33
3.5.2	Diccionarios de Datos	34
3.5.3	Relaciones entre colecciones	46
3.6	IMPLEMENTACIÓN DE LOS MÓDULOS DEL SISTEMA	48
3.6.1	3.6.1 Módulo de Gestión de Procesos	48
3.6.2	3.6.2 Módulo de Usuarios y Roles	50
3.6.3	Módulo de Transacciones	53
3.6.4	Módulo de Reportes	53
3.7	Capa Backend Funcional	54
3.8	Validación y pruebas del sistema	55
3.9	Desarrollo del prototipo funcional	55
3.10	Documentación de Ingeniería Completa	55
	Referencias Bibliográficas	57

Índice de Figuras, Tablas y Diagramas

Figuras

1	Ejemplo de problemas en las cafeterias	2
2	Ejemplo de prueba en el análisis del sistema	26

Tablas

1	Cronograma de Sprints	15
---	---------------------------------	----

Diagramas

1	Diagrama de Arquitectura MERN de tres capas del Sistema POS . . .	13
2	Ejemplo general de Diagrama Estructural UML	28

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS

1.1 Introducción

En el contexto actual de la transformación digital, las organizaciones de todo tamaño enfrentan la necesidad imperiosa de modernizar sus procesos operativos. Los Sistemas de Información Organizacional (SIO) han emergido como la respuesta tecnológica a esta necesidad, permitiendo a las empresas capturar, procesar y distribuir información de manera eficiente, precisa y centralizada. Dentro de esta categoría, los Sistemas de Procesamiento de Transacciones (TPS, por sus siglas en inglés — *Transaction Processing Systems*) representan el eslabón más fundamental: son la capa operativa que registra y procesa cada evento del negocio en tiempo real, constituyendo la base sobre la que se construyen todos los demás niveles de información organizacional.

La evolución histórica de los TPS es un reflejo directo del avance tecnológico. En sus orígenes, durante las décadas de 1960 y 1970, estos sistemas operaban en mainframes centralizados con procesamiento por lotes (*batch processing*), donde las transacciones se acumulaban y procesaban en diferido. Con la llegada de las redes de computadoras y, posteriormente, de Internet, los TPS evolucionaron hacia arquitecturas distribuidas de procesamiento en línea (*OLTP — Online Transaction Processing*), capaces de manejar miles de transacciones concurrentes con tiempos de respuesta de milisegundos. Hoy en día, la adopción de arquitecturas web modernas basadas en *stacks* como MERN (MongoDB, Express.js, React.js, Node.js) ha democratizado la implementación de TPS robustos, haciéndolos accesibles incluso para pequeñas y medianas empresas que anteriormente no podían costear soluciones de este tipo.

En este marco, el presente proyecto propone el desarrollo de un Sistema de Información Organizacional Web basado en el enfoque TPS, orientado específicamente a la gestión integral de una cafetería en la ciudad de La Paz, Bolivia. El sistema, concebido como un Punto de Venta (*Point of Sale — POS*) web, tiene por objetivo digitalizar y centralizar los procesos críticos del negocio: la toma y seguimiento de órdenes por mesa,

la administración del catálogo de productos, el control de acceso diferenciado por roles de usuario, el procesamiento de cobros y la generación automatizada de reportes financieros. La solución busca reemplazar los procesos manuales actualmente vigentes — basados en libretas de papel, cálculos mentales y ausencia total de trazabilidad — por una plataforma tecnológica accesible, segura y escalable que eleve la eficiencia operativa del establecimiento.

1.2 Antecedentes

1.2.1 Antecedentes del objeto de estudio

El objeto de estudio del presente proyecto es una cafetería de tamaño mediano en la ciudad de La Paz, Bolivia, que actualmente opera sus procesos de atención y venta de manera completamente manual. Este escenario, lejos de ser un caso aislado, refleja la realidad predominante en el sector gastronómico local, donde la adopción de tecnologías de gestión sigue siendo incipiente.



Figura 1: Ejemplo de problemas en las cafeterías

En su estado actual, el flujo operativo de la cafetería depende íntegramente de la intervención humana no sistematizada: los pedidos de los clientes son anotados a mano por el personal en libretas o talonarios de papel, la comunicación entre el área de atención y la barra de preparación se realiza de forma verbal o mediante la entrega física de las comandas, y el proceso de cobro se ejecuta mediante cálculos mentales o con

el apoyo de calculadoras de escritorio. Al cierre de cada jornada, el arqueo de caja se realiza de forma manual, contrastando el efectivo físico con las anotaciones del día, un proceso propenso a errores y discrepancias.

Esta situación genera un conjunto de problemas operativos concretos y recurrentes que afectan tanto la eficiencia del servicio como la salud financiera del negocio:

- **Pérdida e ilegibilidad de comandas:** El registro manual en papel está expuesto a extravíos, manchas o escritura ilegible, lo que deriva en errores en la preparación de pedidos y conflictos con los clientes.
- **Ausencia de control y auditoría de usuarios:** Al no existir un sistema de registro, resulta imposible determinar qué miembro del personal procesó, modificó o anuló una orden, eliminando cualquier posibilidad de rendición de cuentas.
- **Descuadres de caja recurrentes:** El cobro basado en cálculo mental o en calculadoras simples, sin un sistema que valide automáticamente los totales, genera discrepancias diarias entre los ingresos registrados y el efectivo real.
- **Nula trazabilidad histórica:** La ausencia de una base de datos implica que no existe ningún registro histórico de ventas. La gerencia no puede conocer cuáles son los productos más vendidos, los picos de demanda por hora o los ingresos acumulados por período, privándose de información crítica para la toma de decisiones estratégicas.

1.2.2 Referencias técnicas de otros sistemas TPS

Como parte del análisis de referencia, se relevaron tres sistemas de gestión para cafeterías y restaurantes disponibles públicamente en GitHub. El estudio de estos proyectos permite identificar patrones de diseño comunes, tecnologías adoptadas por la comunidad y brechas funcionales que el presente sistema busca superar.

1. proyecto-cafeteria — ValentinHer (GitHub)

- **Repositorio:** <https://github.com/ValentinHer/proyecto-cafeteria.git>

- **Descripción general:** Sistema de gestión para cafetería desarrollado como proyecto académico. Implementa las operaciones básicas de un punto de venta: registro de productos, toma de pedidos y generación de órdenes.
- **Stack tecnológico:** Aplicación web con arquitectura cliente-servidor, base de datos relacional para la persistencia de productos y transacciones, e interfaz de usuario orientada a la administración del negocio.
- **Funcionalidades identificadas:** Gestión del catálogo de productos (CRUD), registro de pedidos por mesa, y consulta de historial de ventas a nivel básico.
- **Diferencia con el sistema propuesto:** Este proyecto carece de un sistema de autenticación basado en roles diferenciados (Administrador vs. Cajero), no implementa procesamiento transaccional ACID para garantizar la integridad de los cobros concurrentes, y no genera comprobantes de pago en formato PDF. El sistema propuesto aborda estas limitaciones mediante JWT, sesiones de transacción en MongoDB y el módulo de facturación con PDFKit/jsPDF.

2. Sistema-POS-Restaurantes — ValentinPacheco (GitHub)

- **Repositorio:** <https://github.com/ValentinPacheco/Sistema-POS-Restaurantes.git>
- **Descripción general:** Sistema de Punto de Venta orientado a restaurantes, con enfoque en la gestión de órdenes en sala y el flujo de cobro al cliente. Representa una solución más cercana al dominio del presente proyecto por su naturaleza POS.
- **Stack tecnológico:** Implementación web con separación de capas frontend y backend, manejo de estado de mesas y control de órdenes activas.
- **Funcionalidades identificadas:** Panel de estado de mesas, creación y modificación de órdenes en curso, cálculo de totales y cierre de cuenta por mesa.
- **Diferencia con el sistema propuesto:** Si bien aborda la gestión de

mesas y órdenes, el sistema no contempla una arquitectura de microservicios contenerizada con Docker, ni un despliegue en infraestructura *cloud* (AWS/DigitalOcean). Asimismo, el control de acceso por roles y la generación automatizada de reportes financieros con cortes de caja son funcionalidades ausentes que el presente proyecto incorpora de forma nativa.

3. **cafeteria-app** — FFW4 (GitHub)

- **Repositorio:** <https://github.com/FFW4/cafeteria-app.git>
- **Descripción general:** Aplicación web para la gestión de una cafetería, con foco en la experiencia del operador en el punto de atención. Desarrollada con un enfoque pragmático orientado a la agilidad en la toma de pedidos.
- **Stack tecnológico:** Aplicación de página única (*SPA*) con componentes reactivos para la interfaz del POS y conexión a un servicio de backend para la persistencia de datos.
- **Funcionalidades identificadas:** Selección de productos por categoría, armado del carrito de compras, asignación de pedidos y registro de ventas.
- **Diferencia con el sistema propuesto:** Esta aplicación no implementa autenticación segura mediante *tokens* JWT ni encriptación de contraseñas con *bcrypt*, exponiendo el sistema a vulnerabilidades de acceso no autorizado. Tampoco cuenta con un módulo de reportes analíticos ni con transacciones ACID multi-documento que garanticen la inmutabilidad de los registros históricos de venta, aspectos críticos en un TPS de producción.

1.3 Descripción del objeto de estudio

La unidad de análisis del presente proyecto es una cafetería de servicio rápido ubicada en la ciudad de La Paz, Bolivia, con capacidad para atender simultáneamente a múltiples mesas. El establecimiento ofrece un menú compuesto principalmente por bebidas calientes y frías (café, infusiones, batidos) y una selección de alimentos de preparación rápida (postres, sándwiches, snacks), atendiendo a una clientela diversa

en horario continuo.

Estructura organizacional y actores del sistema:

El personal operativo del establecimiento se organiza en dos roles funcionales claramente diferenciados, que se corresponden directamente con los actores del sistema a desarrollar:

- **Administrador:** Responsable de la gobernanza general del negocio. Gestiona el catálogo de productos (altas, bajas y modificaciones de precios), administra las cuentas del personal operativo y tiene acceso a los reportes de ventas e indicadores financieros.
- **Cajero / Operador POS:** Personal de atención directa al cliente. Su función central es construir y procesar órdenes en la interfaz del punto de venta, asignarlas a la mesa correspondiente y ejecutar el cobro al momento de cerrar la cuenta.

Flujo actual del negocio (situación sin sistema):

El ciclo de servicio actual sigue el siguiente flujo manual: el cliente se ubica en una mesa disponible → el cajero toma el pedido verbalmente y lo anota en la comanda de papel → la comanda se entrega en barra para preparación → los productos son entregados al cliente → al solicitar la cuenta, el cajero suma manualmente los ítems, comunica el total y recibe el pago en efectivo → el dinero se deposita en la caja registradora mecánica.

Flujo propuesto del negocio (con el sistema TPS):

Con la implementación del sistema, el flujo se transforma radicalmente: el cajero selecciona los productos del menú digital interactivo y los asigna a la mesa del cliente en la interfaz POS → el sistema calcula automáticamente el subtotal en tiempo real → al confirmar el cobro, el *backend* procesa matemáticamente la transacción, aplica impuestos y sella el registro de forma inmutable en la base de datos → el sistema genera automáticamente el comprobante de pago (ticket/factura) → la mesa queda marcada como disponible para el próximo cliente. Cada uno de estos eventos queda registrado con fecha, hora, cajero responsable y detalle de productos, garantizando trazabilidad completa y eliminando cualquier posibilidad de descuadre.

1.4 Descripción de Procesos del Sistema

Esta sección describe la forma en que el Sistema TPS-POS será implementado operativamente, detallando los procesos clave que lo componen y cómo cada uno se ejecuta dentro de la arquitectura MERN propuesta.

1.4.1 Proceso 1: Autenticación y Control de Acceso

Al ingresar al sistema, el operador introduce sus credenciales (usuario y contraseña) en la pantalla de *login* construida con React.js. El *frontend* envía una solicitud HTTP POST al *endpoint* `/api/auth/login` del servidor Node.js/Express.js. El *backend* recupera el registro del usuario desde MongoDB, verifica la contraseña mediante la función de comparación de *bcrypt* y, si es válida, genera un JSON Web Token (JWT) firmado que incluye en su *payload* el identificador del usuario y su rol (Administrador o Cajero). Este *token* es devuelto al cliente y almacenado en el estado global de Redux, siendo adjuntado automáticamente en la cabecera *Authorization* de todas las peticiones subsiguientes. Los *middlewares* de Express validan el *token* en cada ruta protegida antes de permitir el acceso a los recursos.

1.4.2 Proceso 2: Gestión del Catálogo (Administrador)

El Administrador accede al módulo de gestión de productos desde su panel exclusivo. A través de formularios React, puede crear, editar, activar o desactivar ítems del menú (nombre, precio, categoría, imagen). Cada acción dispara una petición REST al *backend* (POST, PUT o PATCH según corresponda), que valida los datos contra el esquema Mongoose de la colección *Productos* en MongoDB antes de persistir el cambio. Las modificaciones de precio no alteran registros históricos: las órdenes ya cerradas conservan el precio exacto del momento de la venta gracias a la desnormalización del carrito (*cartItems*).

1.4.3 Proceso 3: Registro de Orden en el POS (Cajero)

El Cajero opera la interfaz POS táctil. Selecciona una mesa disponible del panel de estados, luego elige productos del menú interactivo organizado por categorías. Cada selección actualiza el estado del carrito en Redux, recalculando subtotales y totales en tiempo real sin consultar el servidor. Una vez completada la orden, el Cajero confirma el método de pago. El *frontend* envía la orden completa (mesa, cajero, *cartItems* con precios actuales, total calculado) al *endpoint* `/api/orders` del *backend*.

1.4.4 Proceso 4: Procesamiento Transaccional y Cierre (Backend)

Al recibir la orden, el *backend* inicia una Sesión de Transacción de MongoDB para garantizar las propiedades ACID. Dentro de la transacción atómica se ejecutan en secuencia: (1) validación matemática del total enviado por el cliente, (2) inserción del documento de la orden en la colección Facturas/Órdenes con todos los datos inmutables, (3) actualización del estado de la Mesa asignada de “ocupada” a “disponible”. Si cualquiera de estos pasos falla (por ejemplo, por un corte de red), MongoDB ejecuta un *Rollback* completo, garantizando que no queden “ventas a medias” en la base de datos.

1.4.5 Proceso 5: Generación de Comprobante (Factura/Ticket)

Una vez confirmada la transacción, el *backend* devuelve los datos de la orden sellada al *frontend*. React activa automáticamente la opción de generar el comprobante, invocando el módulo de facturación (PDFKit o jsPDF) que construye el ticket en formato PDF con el detalle de los ítems, el total cobrado, la mesa, el cajero y la fecha exacta. El comprobante queda disponible para impresión inmediata desde el navegador.

1.4.6 Proceso 6: Reportes y Arqueo de Caja

El Administrador accede al módulo de reportes para consultar el resumen financiero del turno o del período seleccionado. El *backend* ejecuta consultas de agregación (*Ag-*

gregation Pipeline) sobre la colección de órdenes en MongoDB, consolidando ingresos totales, desglose por método de pago, productos más vendidos y ventas por cajero. Los resultados son renderizados en tablas y gráficos en el *frontend* de React, permitiendo al Administrador realizar el arqueado de caja con datos exactos y auditables.

1.5 Formulación del Problema

¿De qué manera el desarrollo e implementación de un Sistema de Información Organizacional Web basado en el enfoque de Procesamiento de Transacciones (TPS), construido sobre la arquitectura MERN, puede optimizar la gestión de pedidos, la administración de mesas, el control de usuarios por roles, el procesamiento de cobros y la generación de reportes financieros en una cafetería de la ciudad de La Paz, reduciendo los errores operativos y garantizando la trazabilidad e integridad de cada transacción?

1.6 Objetivos

1.6.1 Objetivo General

Desarrollar un Sistema Web de Punto de Venta (POS) para cafeterías, basado en la arquitectura MERN (MongoDB, Express.js, React.js, Node.js), que permita optimizar la gestión integral de pedidos, la administración visual de mesas y la facturación automatizada, reduciendo los errores operativos y mejorando la experiencia del cliente en establecimientos gastronómicos medianos de la ciudad de La Paz.

1.6.2 Objetivos Específicos

- Implementar un módulo de gestión de pedidos en tiempo real que permita a los meseros registrar, modificar y hacer seguimiento del estado de las órdenes (en preparación, servido, pagado) mediante una interfaz táctil dinámica construida con React.js.
- Diseñar e integrar un sistema de control de acceso basado en roles (RBAC) con

autenticación segura mediante JSON Web Tokens (JWT), diferenciando los permisos y vistas del Administrador, el Cajero y el Mesero.

- Desarrollar un módulo de administración visual de mesas y reservas que presente un panel de estados en tiempo real, permitiendo a los operadores conocer la disponibilidad y el estado de cada mesa del establecimiento.
- Construir las APIs RESTful del *backend* con Node.js y Express.js, conectadas a una base de datos MongoDB, para soportar todas las operaciones CRUD de los módulos de productos, órdenes, mesas y usuarios.
- Implementar un módulo de facturación y generación de recibos que automatice el cálculo del cobro total y produzca comprobantes detallados en formato PDF, integrando métodos de pago simulados mediante una pasarela estándar.
- Desplegar el sistema en infraestructura *cloud* (AWS o DigitalOcean) utilizando contenedores Docker para garantizar la portabilidad, disponibilidad y escalabilidad del entorno productivo.

Nivel 2 — Diagrama de Contenedores (*Containers*): Descompone el sistema en sus grandes bloques tecnológicos desplegables: las aplicaciones, los servicios y los almacenes de datos que lo componen. Para el Sistema POS, este nivel revela los tres contenedores principales de la arquitectura MERN: la **Aplicación Web** (*Single Page Application* en React.js, ejecutada en el navegador del operador), la **API REST** (servidor Node.js/Express.js desplegado en un contenedor Docker en AWS EC2) y la **Base de Datos** (instancia de MongoDB). Este diagrama responde: ¿cómo está estructurado técnicamente el sistema y cómo se comunican sus partes?

Nivel 3 — Diagrama de Componentes (*Components*): Profundiza dentro de un contenedor específico, mostrando los componentes lógicos que lo integran y sus responsabilidades. Para el contenedor de la **API REST**, los componentes serían: el *Router de Autenticación* (gestiona el registro y *login*), el *Middleware JWT* (valida tokens en cada petición), el *Controlador de Órdenes* (maneja el ciclo de vida de los pedidos), el *Controlador de Productos* (gestiona el catálogo del menú), el *Controlador de Mesas*

(administra el estado de las mesas) y el *Generador de Reportes PDF* (produce los comprobantes de venta). Este nivel responde: ¿qué hace cada parte interna del contenedor?

Nivel 4 — Código (Code): Es el nivel más detallado y opcional, representado mediante diagramas de clases UML o directamente a través del código fuente documentado. Para el Sistema POS, este nivel se materializa en los esquemas de *Mongoose* (UserSchema, ProductSchema, OrderSchema, TableSchema) y en la estructura de clases o módulos del *frontend* React. Su propósito es responder: ¿cómo está implementado internamente cada componente?

La adopción del Modelo C4 en este proyecto garantiza que la arquitectura MERN quede documentada de forma consistente y navegable, facilitando la incorporación de nuevos desarrolladores al equipo y la comunicación técnica con el cliente. Los diagramas de Contexto y Contenedores se incluyen en el presente informe; los de Componentes y Código se desarrollan en el Marco Práctico.

1.7 Análisis preliminar del sistema TPS

Como Ingeniero de Requerimientos y Datos, el análisis preliminar dictamina que la transición de una cafetería puramente manual a un entorno digital requiere modelar transacciones rápidas y precisas, integrando la gestión física del local (mesas). Se han especificado los siguientes requerimientos de persistencia y flujo:

- **Detalle de Usuarios (Actores y Control de Acceso):**
 - **Administrador:** Actor con privilegios absolutos (*Full CRUD*). Su rol a nivel de datos implica alimentar las entidades maestras del sistema: gestión del catálogo de ítems de la cafetería (nombres, precios, imágenes, categorías como 'Cafés', 'Postres'), control de inventario base y la creación o inhabilitación de cuentas para el personal operativo.
 - **Cajero / Operador POS:** Actor confinado a la interfaz de Punto de Venta. Su interacción con la base de datos es de lectura sobre el menú y de escritura

intensiva sobre las colecciones/tablas de ventas, sin permisos para alterar precios históricos o borrar registros contables.

- **Detalle de Transacciones (El Flujo de la Orden y Mesas):**

- **Transacción POS y Asignación:** Reemplazando por completo la libreta de papel, el nuevo flujo demanda que el cajero construya la orden mediante un panel interactivo con categorías. El modelo de datos requerirá que, antes de proceder al cobro, la colección del carrito de compras (compuesta por los ítems seleccionados y sus cantidades) sea vinculada obligatoriamente al nombre del cliente y al **número de mesa**.
- **Facturación y Cierre:** Una vez que se confirma el método de pago, el *back-end* debe procesar matemáticamente el subtotal, aplicar los impuestos correspondientes y sellar la transacción con la fecha exacta. El registro resultante en la base de datos se vuelve inmutable y dispara en el *frontend* la opción de generar e imprimir la factura o ticket (*Bill/Receipt*), dando por finalizado el servicio para esa mesa.

1.8 Propuesta de solución

- **Sistema:** Sistema de Punto de Venta (POS) Web enfocado en TPS.
- **Arquitectura:** Patrón Cliente — Servidor con separación de responsabilidades API REST y arquitectura de tres capas (presentación, lógica de negocio y datos) siguiendo el patrón MVC (Modelo-Vista-Controlador).

A continuación se presenta el diagrama de la arquitectura propuesta:

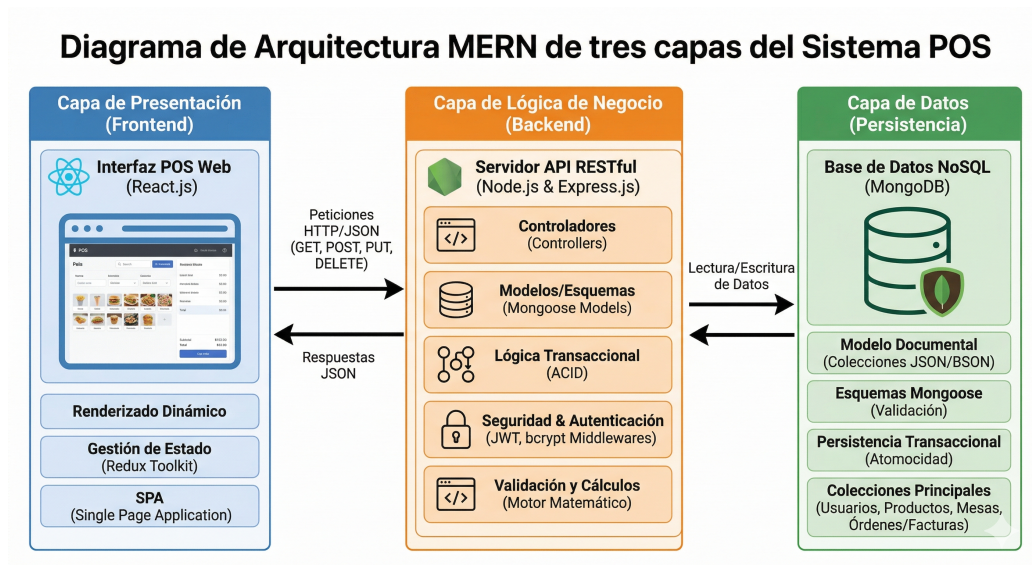


Diagrama 1: Diagrama de Arquitectura MERN de tres capas del Sistema POS

1.9 Cronograma

El proyecto tiene una duración total de **4 meses (16 semanas)**, organizado en 8 *Sprints* de 2 semanas cada uno bajo el marco Scrum.

Sprint	Semanas	Fase	Actividades principales	Entregable
0	1–2	Inicio y Diseño	Levantamiento de requerimientos, diseño de <i>wireframes</i> UI/UX, modelado de base de datos, configuración del repositorio GitHub y entorno Docker.	<i>Product Backlog</i> , diseño de BD, <i>wireframes</i> aprobados.

Sprint	Semanas	Fase	Actividades principales	Entregable
1	3–4	<i>Backend</i> – Fundamentos	Configuración de Express.js, conexión MongoDB con Mongoose, modelos de datos (User, Product, Table, Order), sistema de autenticación JWT y <i>middleware</i> de roles.	API de autenticación funcional (registro, <i>login</i> , roles).
2	5–6	<i>Backend</i> – Módulos Core	APIs RESTful para gestión de productos del menú (CRUD), gestión de mesas (CRUD + estados) y gestión de órdenes (crear, actualizar estado).	<i>Endpoints</i> de Products, Tables y Orders documentados.
3	7–8	<i>Frontend</i> – Base y Auth	Configuración de React.js + Redux Toolkit + React Router, pantallas de <i>Login</i> , <i>layout</i> principal y conexión con el API de autenticación.	<i>Frontend</i> base funcional con <i>login</i> y roles.
4	9–10	<i>Frontend</i> – POS	Módulo POS táctil: selección de categorías, productos, cantidad y adición al carrito de órdenes; envío de pedidos al <i>backend</i> .	Interfaz POS funcional conectada al <i>backend</i> .
5	11–12	Mesas y Dashboard	Panel visual de estados de mesas, módulo de administración de menú (alta/baja de productos), vista de órdenes activas para cocina.	Gestión de mesas e interfaz de cocina operativa.

Sprint	Semanas	Fase	Actividades principales	Entregable
6	13–14	Facturación y Pagos	Módulo de generación de facturas en PDF, cálculo automático de totales, integración de métodos de pago simulados, historial de ventas.	Facturación y cierre de órdenes completo.
7	15–16	Cierre: QA y Despliegue	Pruebas funcionales e integración (QA), corrección de <i>bugs</i> , despliegue en AWS/DigitalOcean con Docker, documentación técnica final y capacitación al usuario.	Sistema desplegado en producción y manual de usuario.

Tabla 1: Cronograma de Sprints

Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS

2.1 Sistemas de Información Organizacional

2.1.1 Definición

Un Sistema de Información Organizacional (SIO) es un conjunto integrado de componentes — personas, procesos, datos, *hardware* y *software* — diseñado para recolectar, almacenar, procesar y distribuir información que apoye la coordinación, el control, el análisis y la toma de decisiones dentro de una organización (Laudon & Laudon, 2020). A diferencia de un simple programa informático, un SIO está profundamente imbricado con los procesos de negocio de la organización: define cómo fluye la información entre los actores, cuándo se captura, cómo se transforma y quién tiene acceso a ella.

En términos más precisos, un SIO transforma datos crudos (ej. el registro de una venta) en información significativa y estructurada (ej. un reporte de ingresos diarios), que a su vez se convierte en conocimiento útil para la gestión (ej. la identificación del turno de mayor rentabilidad). Este proceso de transformación es el núcleo del valor que aportan los SIO a las organizaciones modernas.

2.1.2 Componentes

Todo Sistema de Información Organizacional se articula en torno a seis componentes fundamentales que trabajan de forma interdependiente:

- **Hardware:** La infraestructura física que sustenta el sistema: servidores, terminales de trabajo, dispositivos de red y, en el contexto del presente proyecto, las estaciones de trabajo desde las que el personal operará la interfaz POS.
- **Software:** Los programas y aplicaciones que procesan los datos. Incluye tanto el *software* de sistema (sistema operativo, entorno de ejecución Node.js) como el *software* de aplicación desarrollado a medida (la plataforma POS web).
- **Datos:** La materia prima del sistema. En el contexto de la cafetería, los datos

son las órdenes, los productos, los usuarios, las mesas y las transacciones que el sistema captura y persiste en la base de datos MongoDB.

- **Redes y telecomunicaciones:** La infraestructura de conectividad que permite el acceso concurrente al sistema desde múltiples dispositivos, habilitado por la arquitectura cliente-servidor del proyecto.
- **Procedimientos:** Los protocolos y flujos de trabajo que definen cómo deben interactuar los usuarios con el sistema (ej. el proceso de apertura de turno, la toma de una orden, el cierre de caja).
- **Recursos humanos:** Los actores que operan el sistema. En el proyecto, esto comprende al Administrador y al Cajero, cada uno con roles y permisos claramente delimitados.

2.1.3 Tipos de sistemas

Desde una perspectiva funcional, los SIO se clasifican en distintos tipos según el nivel organizacional al que sirven. Los **Sistemas de Procesamiento de Transacciones (TPS)** operan en el nivel operativo, capturando y procesando las transacciones cotidianas del negocio. Los **Sistemas de Información Gerencial (MIS)** consolidan la información del TPS para generar reportes estructurados destinados a la gerencia media. Los **Sistemas de Soporte a Decisiones (DSS)** asisten en la toma de decisiones complejas mediante análisis de datos y modelos. Los **Sistemas de Información Ejecutiva (EIS)** proveen información estratégica de alto nivel a los directivos. El presente proyecto se enfoca en la implementación de un TPS, que actúa como la base de toda esta pirámide informacional.

2.2 Sistema de Procesamiento de Transacciones (TPS)

2.2.1 Definición

Un Sistema de Procesamiento de Transacciones es un tipo especializado de SIO diseñado para capturar, procesar, validar y almacenar las transacciones operativas de una organización de forma inmediata, confiable y a gran escala (O'Brien & Marakas, 2011).

En el contexto del negocio, una **transacción** se define como cualquier evento discreto que modifica el estado de los datos del sistema y que debe quedar registrado de forma permanente e inalterable: el registro de una venta, la creación de una orden, el cobro de una cuenta o la modificación del catálogo de productos.

2.2.2 Características

Los TPS se distinguen de otros tipos de sistemas de información por un conjunto de atributos técnicos y funcionales que los hacen aptos para el procesamiento operativo de alto volumen:

- **Procesamiento en tiempo real (OLTP):** A diferencia del procesamiento por lotes (*batch*), los TPS modernos procesan cada transacción en el instante en que se produce, actualizando la base de datos de forma inmediata y reflejando el estado actual del negocio en todo momento.
- **Alta confiabilidad y disponibilidad:** Un TPS para un punto de venta debe estar disponible durante todo el horario operativo del negocio. La indisponibilidad del sistema implica la parálisis del servicio al cliente.
- **Integridad transaccional (propiedades ACID):** Toda transacción en un TPS debe cumplir las propiedades de Atomicidad (la transacción se ejecuta completa o no se ejecuta), Consistencia (el sistema pasa de un estado válido a otro estado válido), Aislamiento (las transacciones concurrentes no interfieren entre sí) y Durabilidad (una transacción confirmada persiste incluso ante fallos del sistema) (Elmasri & Navathe, 2015).
- **Manejo de alto volumen de datos estandarizados:** Los TPS están optimizados para procesar grandes cantidades de transacciones simples y repetitivas de forma eficiente, a diferencia de los sistemas analíticos que trabajan con consultas complejas sobre datos históricos.

2.2.3 Funciones

En el contexto específico del presente proyecto, el TPS ejecuta el siguiente ciclo funcional para cada transacción de venta:

1. **Captura de datos de origen:** El cajero construye la orden seleccionando productos del catálogo digital y asignándola a una mesa, introduciendo los datos de la transacción en el sistema mediante la interfaz POS de React.js.
2. **Validación y verificación:** El *backend* (Node.js/Express.js) verifica que el usuario tenga los permisos necesarios (validación JWT), que los ítems existan en el catálogo activo y que la mesa esté disponible.
3. **Procesamiento matemático:** El motor transaccional calcula automáticamente los subtotales por ítem, aplica los impuestos correspondientes y determina el total a cobrar, eliminando el margen de error del cálculo manual.
4. **Actualización de la base de datos:** La transacción se escribe de forma atómica en MongoDB, vinculando el documento de la orden con el cajero responsable, la mesa asignada y los ítems del carrito con sus precios exactos en ese instante.
5. **Emisión del comprobante:** El sistema genera el ticket o factura en formato PDF, disponible para impresión inmediata, y actualiza el estado de la mesa a “disponible”.

2.2.4 Evolución hacia sistemas web

Los TPS han recorrido un largo camino desde las terminales monolíticas de los años setenta. La adopción de arquitecturas web modernas —como la empleada en este proyecto— representa la fase más reciente de esta evolución, caracterizada por tres ventajas fundamentales: **ubicuidad** (el sistema es accesible desde cualquier dispositivo con navegador web en la red local del negocio), **centralización** (todos los datos residen en un único repositorio en la nube, eliminando la dispersión de información), y **escalabilidad** (la arquitectura basada en microservicios y contenedores Docker permite escalar el sistema horizontalmente para absorber incrementos en la carga de trabajo sin rediseñar la arquitectura base).

2.3 Arquitectura de sistemas web

La arquitectura del sistema POS se basa en el modelo Cliente–Servidor, una de las estructuras más utilizadas en aplicaciones web modernas por su capacidad de separación de responsabilidades, escalabilidad y mantenimiento (Sommerville, 2015).

2.3.1 Cliente

El cliente representa la capa de presentación del sistema, encargada de interactuar directamente con el usuario final mediante una interfaz gráfica accesible desde el navegador. En este proyecto, el cliente será desarrollado utilizando React.js, permitiendo:

- Renderizado dinámico de componentes (Virtual DOM).
- Interacciones en tiempo real en el POS.
- Manejo del estado global mediante Redux Toolkit.
- Navegación sin recarga de página (SPA).

Funciones principales:

- Capturar datos de entrada (pedidos, login).
- Mostrar información procesada.
- Enviar solicitudes HTTP al servidor.

2.3.2 Servidor

El servidor constituye la capa de lógica de negocio. Tecnologías: Node.js; Express.js.

Funciones principales:

- Procesamiento de órdenes.
- Validaciones y cálculos.
- Ejecución de la lógica transaccional.

2.3.3 API

La API permite la comunicación entre cliente y servidor mediante HTTP y JSON. Actúa como el puente documentado que estructura y transmite la información bidireccionalmente. Características:

- Métodos estándar: GET, POST, PUT, DELETE.
- Arquitectura RESTful.

2.3.4 Base de datos

Repositorio central donde reposa la persistencia de las entidades. Es el componente responsable de almacenar los datos operacionales a largo plazo para asegurar la durabilidad y disponibilidad de la información de las ventas y el menú.

2.3.5 Flujo del Sistema

1. Usuario interactúa con frontend.
2. Cliente envía petición HTTP a la API.
3. Servidor procesa la solicitud y valida.
4. Se consulta o impacta la base de datos.
5. Servidor responde en JSON.
6. Frontend actualiza la interfaz.

2.4 Seguridad en sistemas de información

Como modelador y encargado de la seguridad arquitectónica, se establece que un sistema de ventas (POS) debe proteger de forma absoluta sus *endpoints* (rutas de API) y la persistencia de datos.

2.4.1 Autenticación

Se descartan las sesiones tradicionales. El sistema implementa JSON Web Tokens (JWT). Tras validar credenciales (contraseñas previamente procesadas con funciones

criptográficas unidireccionales de *hash*, como *bcrypt*), el *backend* emite un token firmado (Stallings, 2017). Este token viaja en las cabeceras HTTP de cada petición del cliente, garantizando que el usuario es quien dice ser sin consultar la base de datos reiteradamente.

2.4.2 Autorización

La autorización asegura que un usuario autenticado solo pueda realizar las acciones para las que está facultado. Se ejecuta verificando los niveles de privilegio incrustados y firmados criptográficamente en el token antes de responder a una petición.

2.4.3 Roles

El modelo de datos incluye una propiedad rígida de “Rol” (ej. Administrador, Cajero). Este mecanismo se implementa mediante *Middlewares* (bloques de código intermedios en el *backend*) que desenscriptan el *payload* del token y rechazan con un error 403 (Prohibido) cualquier intento de un Cajero de acceder a las rutas de eliminación de usuarios o reportes gerenciales.

2.4.4 Control de acceso

Tanto a nivel de la interfaz (ocultando botones de configuración a cajeros) como a nivel de capa de datos, se aplican políticas estrictas para evitar inyecciones maliciosas o robo de sesiones, blindando el flujo desde que se presiona “Cobrar” hasta que la información reposa en el disco.

2.5 Base de datos

El diseño de la persistencia de datos constituye el corazón del sistema, siendo responsabilidad directa de la ingeniería de datos modelar la información de la cafetería para que sea escalable, rápida y matemáticamente exacta.

2.5.1 Modelo relacional

Aunque tecnologías modernas como la pila MERN utilicen modelos NoSQL orientados a documentos para optimizar la velocidad transaccional, los principios lógicos relacionales son ineludibles en un TPS (Elmasri & Navathe, 2015). Las entidades maestras se identifican y separan claramente: Usuarios (Personal), Categorías (Clasificación del menú), Productos (Ítems de venta) y Facturas/Órdenes (Registro de la transacción). Se definen llaves referenciales explícitas entre ellas para establecer cardinalidad (ej. un cajero realiza muchas ventas, una orden contiene múltiples productos).

2.5.2 Integridad

Los principios lógicos de integridad se mantienen ineludibles mediante el uso de esquemas de validación estrictos (como *Mongoose* en el caso de la pila seleccionada). Estos esquemas garantizan la exactitud de los tipos de datos ingresados y previenen la inserción de documentos huérfanos o con información financiera incompleta.

2.5.3 Normalización

Se aplican reglas de normalización para evitar anomalías; por ejemplo, la información del perfil del usuario o la descripción detallada de un producto no se repiten innecesariamente. Sin embargo, por requerimientos de diseño de un POS y para proteger la contabilidad, se realiza una desnormalización controlada en las Órdenes: al registrar una venta, el precio exacto actual del producto se copia de forma fija dentro de la orden. Esto garantiza que la información histórica sea inmutable frente a futuros cambios de precios en el catálogo.

2.5.4 Transacciones

En el entorno TPS, una transacción es indivisible. Registrar una venta implica: calcular totales, insertar la orden, asociar el método de pago y actualizar la disponibilidad de la mesa. El motor de la base de datos se configura para garantizar atomicidad (Principios

ACID), asegurando que si ocurre un fallo de red a la mitad del proceso, la base de datos ejecute un *Rollback* (reversión completa de los pasos previos), previniendo que existan “ventas a medias” o corrupciones en los arcos de caja.

2.6 Metodología de desarrollo

2.6.1 Scrum

Es un marco de trabajo ágil para el desarrollo, entrega y mantenimiento de productos complejos, definido en la *Scrum Guide* (Schwaber & Sutherland, 2020). Se fundamenta en tres pilares empíricos: transparencia, inspección y adaptación. Para el Sistema POS de Cafetería, Scrum es la elección metodológica óptima porque permite iterar rápidamente y ajustar requerimientos de acuerdo a la retroalimentación del cliente.

Roles

- **Product Owner:** Es el responsable de maximizar el valor del producto. Sus funciones son definir y mantener el *Product Backlog*; ser el punto de contacto único con el cliente; y aceptar o rechazar los incrementos funcionales.
- **Scrum Master:** Responsable de que el equipo aplique correctamente Scrum. Facilita las ceremonias, elimina impedimentos y protege al equipo de interrupciones externas.
- **Equipo de Desarrollo:** Equipo autoorganizado responsable de convertir los ítems del *Product Backlog* en un incremento potencialmente entregable (programación, diseño y pruebas).

Artefactos

- **Product Backlog:** Lista única y priorizada de todos los requerimientos funcionales y técnicos necesarios para el sistema.
- **Sprint Backlog:** Conjunto de requerimientos seleccionados para el *Sprint* actual, divididos en tareas concretas a desarrollar por el equipo.

- **Incremento:** La suma de todas las funcionalidades completadas durante el *Sprint*, las cuales deben cumplir con la Definición de Hecho (código probado y validado).

Eventos

- **Sprint Planning (Planificación):** Reunión de inicio donde el equipo define qué se va a entregar y cómo se va a construir el incremento durante el ciclo de trabajo.
- **Daily Scrum (Reunión diaria):** Reunión breve de sincronización del equipo de desarrollo para evaluar el progreso y exponer bloqueos u obstáculos.
- **Sprint Review (Revisión):** Demostración del *software* funcional al *Product Owner* y partes interesadas al finalizar el *Sprint* para recoger impresiones.
- **Sprint Retrospective (Retrospectiva):** Espacio de mejora continua donde el equipo reflexiona sobre sus propios procesos de trabajo de cara a la siguiente iteración.

Capítulo 3. MARCO PRÁCTICO — DESARROLLO DEL SISTEMA TPS

3.1 Análisis del sistema

En base a la recopilación de datos, la estructura organizacional del sistema identifica diversos actores: Administradores, quienes tienen control global; y Operadores, quienes ejecutan diariamente transacciones asociadas a los procesos de negocio. El flujo de procesos demanda digitalizar desde el ingreso del actor al sistema, registro de eventos, hasta la confirmación e impacto histórico.



Figura 2: Ejemplo de prueba en el análisis del sistema

3.2 Determinación de requerimientos

3.2.1 Requerimientos funcionales

Los requerimientos funcionales expresan lo que el sistema **debe hacer** operativamente.

- **RF01:** El sistema permitirá registrar nuevos usuarios operativos.
- **RF02:** El sistema permitirá a los usuarios iniciar y cerrar sesión de manera segura.
- **RF03:** El sistema permitirá al administrador asignar o revocar roles de acceso.
- **RF04:** El sistema permitirá registrar un nuevo proceso organizacional básico.
- **RF05:** El sistema permitirá el registro consecutivo de transacciones asociadas a un proceso.
- **RF06:** El sistema permitirá generar reportes tabulados basados en un rango de fechas.

3.2.2 Requerimientos no funcionales

Establecen las restricciones y la forma en cómo debe operar y comportarse estructuralmente la aplicación.

- **Seguridad:** Encriptación de contraseñas usando algoritmos seguros (ej. *bcrypt*) y comunicaciones seguras.
- **Rendimiento:** Tiempos de respuesta para guardado de transacciones menores a 2 segundos en carga moderada.
- **Usabilidad:** Interfaces intuitivas, adaptables a monitores de escritorio (diseño responsivo).
- **Disponibilidad:** Arquitectura preparada para mantener disponibilidad en un entorno de servidor las 24 horas.
- **Escalabilidad:** Separación modular del código que facilite agregar futuros módulos sin modificar el núcleo operativo.

3.3 Modelado del sistema

3.3.1 Historias de Usuario

Se definen detallando la necesidad y la regla de aceptación:

- *Como* administrador, *quiero* registrar y eliminar usuarios *para* controlar estrictamente el acceso a la plataforma corporativa.
- *Como* operador de sistemas, *quiero* registrar una transacción en tiempo de ejecución *para* avanzar en mi cuota diaria de procesos.

3.3.2 Diagramas UML

A continuación se muestra un ejemplo genérico de la estructura de un diagrama, en este caso, se deben incrustar aquí los diagramas correspondientes: Diagramas de Casos de Uso, Clases, Secuencia y Actividades generados en herramientas tipo StarUML o Drawio.



Diagrama 2: Ejemplo general de Diagrama Estructural UML

3.4 3.4 Diseño del sistema

3.4.1 Arquitectura del sistema — Modelo C4

El diseño arquitectónico del Sistema POS para la cafetería se documenta utilizando el **Modelo C4** (*Context, Containers, Components, Code*), un estándar de representación jerárquica que permite comunicar la arquitectura de software a diferentes audiencias —desde la gerencia hasta los desarrolladores— con el nivel de detalle apropiado para cada una (Brown, 2018).

Nivel 1 — Diagrama de Contexto (*System Context*) El Diagrama de Contexto es la vista de más alto nivel. Su propósito es mostrar el sistema como una caja negra y situar al lector en el entorno donde opera: quiénes interactúan con él y con qué sistemas externos se conecta.

Actores (usuarios del sistema):

- **Administrador:** Interactúa con el sistema a través de un navegador web en su estación de trabajo. Sus acciones se centran en la gestión del catálogo de productos y usuarios, y en la consulta de reportes financieros.
- **Cajero / Operador POS:** Interactúa con el sistema a través de la interfaz táctil del POS desde la pantalla del punto de atención. Registra órdenes, gestiona el estado de las mesas y procesa los cobros de cada turno.

El sistema central:

- **Sistema POS Web — Cafetería (La Paz, Bolivia):** Plataforma web construida sobre la arquitectura MERN, que centraliza la gestión de transacciones, usuarios, productos y mesas del establecimiento.

Sistemas externos:

- **Pasarela de Pagos (Razorpay / Simulada):** Sistema externo de procesamiento de cobros electrónicos con el que el módulo de pagos del POS se comunica para registrar y confirmar transacciones con tarjeta o QR.

- **Servicio de Hosting Cloud (AWS EC2 / DigitalOcean):** Infraestructura de nube donde se despliegan los contenedores Docker que alojan la API y la base de datos del sistema.

Relaciones clave en este nivel:

El *Administrador* y el *Cajero* acceden al Sistema POS a través del protocolo HTTPS desde sus respectivos navegadores. El Sistema POS se comunica con la *Pasarela de Pagos* mediante llamadas HTTP/REST para procesar cobros electrónicos, y reside desplegado en el *Servicio de Hosting Cloud*.

Nivel 2 — Diagrama de Contenedores (*Containers*) El Diagrama de Contenedores descompone el sistema en sus bloques tecnológicos desplegados de forma independiente. Cada contenedor es una unidad ejecutable (una aplicación, un servicio, una base de datos) con una tecnología concreta.

Contenedor 1 — Aplicación Web SPA (Frontend)

Atributo	Detalle
Tecnología	React.js 18 + Redux Toolkit + React Router DOM
Tipo	Single Page Application (SPA) — ejecutada en el navegador
Responsabilidad	Renderizar la interfaz del POS, el panel de administración y los reportes. Gestionar el estado global de la sesión y del carrito de compras.

Atributo	Detalle
Comunicación	Envía peticiones HTTP/REST en formato JSON a la API Backend a través de axios. Recibe el JWT del backend y lo adjunta en la cabecera Authorization de cada petición subsiguiente.

Contenedor 2 — API REST (Backend)

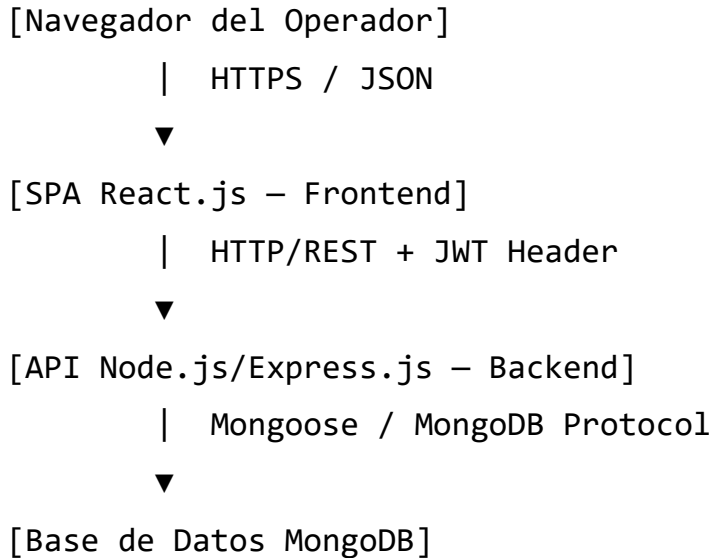
Atributo	Detalle
Tecnología	Node.js 20 LTS + Express.js 4
Tipo	Servidor de API RESTful — desplegado en contenedor Docker (AWS EC2)
Responsabilidad	Exponer los <i>endpoints</i> REST (/api/auth, /api/users, /api/products, /api/tables, /api/orders, /api/payments). Ejecutar la lógica de negocio, validaciones, cálculos transaccionales y control de acceso por roles mediante <i>middlewares</i> JWT.

Atributo	Detalle
Comunicación	Recibe peticiones HTTPS del Frontend. Lee y escribe documentos en MongoDB a través del ODM Mongoose. Invoca la API de la Pasarela de Pagos externa cuando se procesa un cobro electrónico.

Contenedor 3 — Base de Datos (MongoDB)

Atributo	Detalle
Tecnología	MongoDB 7 (desplegado en contenedor Docker o MongoDB Atlas)
Tipo	Base de datos NoSQL orientada a documentos
Responsabilidad	Persistir de forma duradera todos los documentos del sistema: usuarios (users), productos (products), mesas (tables), órdenes (orders) y pagos (payments).
Comunicación	Solo es accedida directamente por el Backend API a través del driver Mongoose. No expone puertos públicos; es accesible únicamente dentro de la red privada del entorno Docker.

Flujo de comunicación entre contenedores:



La separación en tres contenedores independientes garantiza que el Frontend pueda actualizarse sin afectar la API, y que la Base de Datos pueda migrarse a MongoDB Atlas sin modificar la lógica del Backend, favoreciendo la mantenibilidad y escalabilidad del sistema.

3.5 3.5 Diseño de la Base de Datos

3.5.1 Paradigma de persistencia

El sistema adopta **MongoDB** como motor de base de datos NoSQL orientado a documentos, gestionado a través del ODM (Object Document Mapper) **Mongoose**. Esta elección responde a los requisitos de un TPS moderno: flexibilidad en el esquema para los ítems de la orden (cuyo número varía por transacción), alta velocidad de escritura para el registro de ventas en tiempo real, y soporte nativo para transacciones ACID multi-documento mediante *Sessions* en MongoDB 4+.

A pesar del paradigma documental, el diseño lógico preserva los principios de integridad referencial relacionales mediante el uso de `ObjectId` y la configuración `ref` de Mongoose, que permiten realizar operaciones de *populate* (equivalentes a JOIN) entre colecciones relacionadas.

3.5.2 Diccionarios de Datos

Colección: users Almacena los registros del personal operativo y administrativo con acceso al sistema.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
_id	ObjectId	Sí	Identificador único del documento.	Generado automáticamente por MongoDB.
name	String	Sí	Nombre completo del usuario.	Mínimo 3 caracteres.
email	String	Sí	Dirección de correo electrónico.	Validado con expresión regular. Indexado como unique: true.
phone	Number	Sí	Número de teléfono de contacto.	Debe contener 10 dígitos.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
password	String	Sí	Contraseña de acceso al sistema.	Almacenada exclusivamente como <i>hash</i> irreversible generado con Bcrypt (factor de coste: 10). Nunca se persiste en texto plano.
role	String	Sí	Rol funcional asignado al usuario.	Valores admitidos: "Admin" o "Cashier". Determina los permisos de acceso a los <i>endpoints</i> de la API.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
createdAt	Date	Sí	Fecha y hora de creación del registro.	Generado automáticamente por la opción timestamps de Mongoose.
updatedAt	Date	Sí	Fecha y hora de la última modificación.	Actualizado automáticamente por la opción timestamps de Mongoose.

Colección: tables Gestiona la información de las mesas físicas del establecimiento y su estado operativo en tiempo real.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
_id	ObjectId	Sí	Identificador único del documento.	Generado automáticamente por MongoDB.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
tableNo	Number	Sí	Número identificador de la mesa.	Configurado como unique: true. No pueden existir dos mesas con el mismo número.
status	String	No	Estado operativo actual de la mesa.	Valor por defecto: "Avail- able". Valores posibles: "Avail- able" / "Occupied".
seats	Number	Sí	Capacidad máxima de personas de la mesa.	Número entero positivo. Ej: 4, 6.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
currentOrder	ObjectId	No	Referencia al pedido activo asignado a la mesa.	Clave foránea lógica → referencia al documento <code>_id</code> de la colección <code>orders</code> . Valor <code>null</code> cuando la mesa está disponible.

Colección: payments Registra los comprobantes de las transacciones financieras procesadas, almacenando los identificadores de seguimiento de la pasarela de pagos y el estado de cada operación.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
<code>_id</code>	ObjectId	Sí	Identificador único del documento.	Generado automáticamente por MongoDB.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
paymentId	String	No	Identificador de la transacción en la pasarela externa.	Proporcionado por Razorpay o la pasarela configurada (ej. pay_Abc123XYZ).
orderId	String	No	Identificador del pedido asociado al pago.	Relación lógica con la colección orders. Almacenado como String para compatibilidad con IDs de pasarela.
amount	Number	No	Monto total de la transacción.	Valor numérico decimal. Representa el importe cobrado (con impuestos).

Campo	Tipo	Requerido	Descripción	Notas Técnicas
currency	String	No	Código de la moneda de la transacción.	Ej: "BOB" (Boliviano), "USD", "ARS".
status	String	No	Estado de la operación de pago.	Valores posibles: "Captured", "Pending", "Failed", "Refunded".
method	String	No	Canal o instrumento de pago utilizado.	Ej: "Cash", "Credit Card", "QR", "Razorpay".
email	String	No	Correo electrónico del pagador.	Utilizado para el envío del comprobante digital.
contact	String	No	Dato de contacto adicional del pagador.	Número de teléfono u otro identificador de contacto.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
createdAt	Date	No	Fecha y hora de registro del pago.	Registrado manualmente mediante <code>Date.now()</code> al momento de procesar el cobro.

Colección: orders Constituye el eje central del sistema TPS. Registra cada transacción de venta de forma integral e inmutable, vinculando los datos del cliente, los productos consumidos, el detalle de facturación, la mesa asignada y el método de pago.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
_id	ObjectId	Sí	Identificador único del documento.	Generado automáticamente por MongoDB.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
customerDetails	Object	Sí	Datos del cliente para quien se abre el pedido.	Objeto anidado con los sub-campos: name (String), phone (Number) y guests (Number).
orderStatus	String	Sí	Estado actual del pedido en el flujo de servicio.	Valores posibles: "Pending", "In Preparation", "Served", "Paid".
orderDate	Date	No	Fecha y hora de creación del pedido.	Valor por defecto: Date.now().

Campo	Tipo	Requerido	Descripción	Notas Técnicas
bills	Object	Sí	Resumen de facturación calculado por el backend.	Objeto anidado con los sub-campos: total (Number, subtotal sin impuesto), tax (Number, porcentaje de impuesto) y totalWithTax (Number, monto final a cobrar).

Campo	Tipo	Requerido	Descripción	Notas Técnicas
items	Array	No	Lista de productos incluidos en el pedido.	Arreglo flexible de objetos. Cada elemento contiene el detalle del ítem (nombre, precio unitario, cantidad) desnormalizado al momento de la venta para inmutabilidad histórica.
table	ObjectId	No	Mesa física asignada al pedido.	Clave foránea lógica → referencia al documento <code>_id</code> de la colección <code>tables</code> .

Campo	Tipo	Requerido	Descripción	Notas Técnicas
paymentMethodString		No	Método de pago con el que se cerró el pedido.	Ej: "Cash", "Razorpay".
paymentData	Object	No	Datos de confirmación retornados por la pasarela.	Objeto con los IDs de seguimiento de la transacción electrónica (ej. razor-pay_payment_id, razor-pay_order_id).
createdAt	Date	Sí	Fecha y hora de registro del documento.	Generado automáticamente por la opción timestamps de Mongoose.

Campo	Tipo	Requerido	Descripción	Notas Técnicas
updatedAt	Date	Sí	Fecha y hora de la última modificación del documento.	Actualizado automáticamente por la opción timestamps de Mongoose.

3.5.3 Relaciones entre colecciones

El modelo de datos, aunque documental (NoSQL), establece relaciones lógicas explícitas entre las colecciones mediante referencias `ObjectId`, preservando la integridad referencial del sistema TPS:

Relación	Cardinalidad	Descripción
users → orders	1 : N	Un usuario (cajero) puede haber procesado múltiples órdenes a lo largo de su operación. Cada orden registra implícitamente al operador responsable de la sesión activa.

Relación		Cardinalidad	Descripción
tables → orders	1 : N		Una mesa puede estar asociada a múltiples órdenes a lo largo del tiempo (una por cada servicio). En un momento dado, solo una orden puede estar activa por mesa (currentOrder).
tables → orders (activa)	1 : 1		En tiempo real, la relación entre una mesa y su pedido en curso es uno a uno: el campo currentOrder en Table apunta a exactamente un único documento activo en orders.
orders → payments	1 : 1		Cada orden pagada genera exactamente un registro de pago en la colección payments. La relación se establece a través del campo orderId en Payment.

3.6 IMPLEMENTACIÓN DE LOS MÓDULOS DEL SISTEMA

3.6.1 3.6.1 Módulo de Gestión de Procesos

Descripción y propósito El Módulo de Gestión de Procesos constituye el **cimiento operativo** del Sistema POS. Su responsabilidad es la administración de las **entidades maestras** del negocio: el catálogo de **Productos** (el menú de la cafetería) y el inventario de **Mesas** (la infraestructura física del establecimiento). Sin datos correctamente cargados en estas dos entidades, el módulo transaccional del POS no puede operar: no es posible construir una orden sin productos en el menú, ni asignarla a una mesa si estas no están registradas en el sistema.

El acceso a este módulo está **restringido exclusivamente al rol Administrador**, ya que la creación, modificación o eliminación de estos datos maestros afecta directamente la operativa de todos los cajeros del turno.

Submódulo: Gestión del Catálogo de Productos Este submódulo provee las interfaces y los *endpoints* necesarios para mantener actualizado el menú digital de la cafetería. Implementa las cuatro operaciones CRUD completas sobre la colección `products` de MongoDB:

- **Creación (Create):** El Administrador accede al formulario de alta de producto en el panel de administración de React. Ingresar los atributos del ítem (nombre, descripción, precio, categoría —ej. "Cafés", "Infusiones", "Postres", "Snacks"— e imagen). El *frontend* envía una petición `POST /api/products` al backend, que valida los datos contra el esquema Mongoose del `ProductSchema` antes de insertar el nuevo documento en MongoDB.
- **Lectura (Read):** La interfaz POS del cajero consume el *endpoint* `GET /api/products` para cargar el catálogo de productos activos, organizados por categoría, que se muestran en la grilla de selección táctil. El Administrador

puede consultar el listado completo, incluyendo productos inactivos, desde el panel de gestión.

- **Actualización (Update):** El Administrador puede modificar cualquier atributo de un producto existente (incluyendo su precio) mediante una petición `PUT /api/products/:id`. **Restricción clave de integridad histórica:** la actualización de precios solo afecta a futuras órdenes. Los documentos de órdenes ya cerradas preservan el precio exacto del momento de la venta, gracias a la desnormalización controlada del campo `items` en la colección `orders`.
- **Eliminación lógica (Delete):** En lugar de borrar físicamente el registro, el sistema implementa una **eliminación lógica** mediante un cambio de estado (`active: false`). Esto garantiza que los productos que ya forman parte del historial de ventas sigan siendo referenciables en los reportes, sin aparecer en el menú activo del POS.

Submódulo: Gestión de Mesas Este submódulo administra el registro de las mesas físicas del establecimiento, que son los nodos de anclaje del flujo transaccional del POS. Opera sobre la colección `tables` de MongoDB:

- **Alta de mesa (Create):** El Administrador registra una nueva mesa especificando su número (`tableNo`, único en el sistema) y su capacidad en asientos (`seats`). La mesa se crea con estado inicial "Available" y sin orden activa (`currentOrder: null`). Endpoint: `POST /api/tables`.
- **Consulta del panel de mesas (Read):** Los cajeros y el Administrador acceden al *endpoint* `GET /api/tables` para renderizar el panel visual de estados, que muestra en tiempo real qué mesas están disponibles ("Available") y cuáles están ocupadas con un pedido en curso ("Occupied").
- **Actualización de estado (Update):** El estado de una mesa es actualizado automáticamente por el backend durante el flujo transaccional: se marca como "Occupied" al abrir una orden, y vuelve a "Available" al confirmar el pago y cerrar

la transacción. El Administrador también puede actualizar manualmente el estado o los datos de una mesa a través de PUT `/api/tables/:id`.

- **Baja de mesa (Delete):** La eliminación de una mesa del registro solo está permitida si no tiene una orden activa vinculada (`currentOrder: null`), preservando la integridad referencial del historial.
-

3.6.2 3.6.2 Módulo de Usuarios y Roles

Descripción y propósito El Módulo de Usuarios y Roles constituye la **barrera de seguridad** del Sistema TPS. Su función es garantizar que cada actor que interactúe con el sistema sea quien dice ser (*autenticación*) y que solo pueda ejecutar las acciones para las que tiene autorización según su rol (*autorización*). Este módulo implementa un modelo de **Control de Acceso Basado en Roles** (RBAC — *Role-Based Access Control*), diferenciando dos perfiles funcionales: "Admin" y "Cashier".

Mecanismo de autenticación: JSON Web Tokens (JWT) El sistema descarta el uso de sesiones basadas en servidor (*server-side sessions*) —que requieren almacenamiento de estado en el backend y presentan problemas de escalabilidad horizontal— en favor de **autenticación sin estado (*stateless*) mediante JWT** (Jones et al., 2015).

El flujo de autenticación opera de la siguiente manera:

1. **Solicitud de acceso:** El operador ingresa su correo y contraseña en la pantalla de *login* de React. El *frontend* envía una petición POST `/api/auth/login` al backend con las credenciales en el cuerpo del *request*.
2. **Verificación de identidad:** El backend localiza el documento del usuario en la colección `users` a partir del correo electrónico. Utilizando la función `bcrypt.compare()`, compara la contraseña recibida (texto plano) con el *hash* almacenado en la base de datos. Gracias al diseño de Bcrypt, esta comparación es segura incluso ante ataques de fuerza bruta, ya que el proceso de *hashing*

con un factor de coste de 10 hace computacionalmente costosa la verificación masiva de contraseñas.

3. **Emisión del token:** Si las credenciales son válidas, el backend genera un **JSON Web Token** firmado con una clave secreta (JWT_SECRET) almacenada como variable de entorno. El *payload* del token contiene el `_id` del usuario y su role ("Admin" o "Cashier"), con un tiempo de expiración configurado (ej. "8h", equivalente a un turno de trabajo).
4. **Uso del token:** El *frontend* almacena el JWT en el estado global de Redux (y opcionalmente en `localStorage`). A partir de ese momento, **cada petición HTTP al backend incluye el token en la cabecera** `Authorization: Bearer <token>`.
5. **Validación en cada petición:** Un *middleware* de Express (`verifyToken`) intercepta todas las rutas protegidas antes de ejecutar el controlador correspondiente. Deserializa el token usando `jwt.verify()`, extrae el *payload* y lo adjunta al objeto `req.user`, haciendo disponibles la identidad y el rol del solicitante para los *middlewares* de autorización subsiguientes.

Mecanismo de protección de contraseñas: Bcrypt El almacenamiento de contraseñas en texto plano es una vulnerabilidad crítica inaceptable en cualquier sistema de producción. El sistema implementa **Bcrypt** como función de *hashing* adaptativa de contraseñas (Provos & Mazières, 1999):

- **Proceso de registro:** Al crear un nuevo usuario, el backend ejecuta `bcrypt.hash(password, 10)` antes de persistir el documento en MongoDB. Este proceso: (a) genera un *salt* criptográficamente aleatorio, (b) concatena el *salt* con la contraseña, y (c) aplica el algoritmo Blowfish iterativamente $2^{10} = 1.024$ veces, produciendo un *hash* de 60 caracteres que incluye el *salt* incrustado. La contraseña original nunca se almacena ni se puede recuperar.
- **Factor de coste adaptativo:** El valor 10 representa el factor de coste (número de

rondas de *hashing*). Este parámetro puede incrementarse en futuras versiones del sistema para compensar el aumento de la potencia computacional, manteniendo el algoritmo resistente a ataques por diccionario y fuerza bruta sin modificar el código.

Diferenciación de permisos por rol Los dos roles del sistema tienen accesos claramente delimitados, implementados mediante *middlewares* de autorización en el backend y renderizado condicional en el frontend:

Acción / Recurso	Rol Admin	Rol Cashier
Iniciar sesión	SI	SI
Operar el módulo POS (crear y cerrar órdenes)	SI	SI
Consultar estado de mesas	SI	SI
Gestionar catálogo de productos (CRUD)	SI	NO
Gestionar mesas (CRUD)	SI	NO
Crear, editar o eliminar usuarios	SI	NO
Acceder al módulo de reportes financieros	SI	NO
Consultar historial completo de ventas	SI	NO
Ver únicamente las ventas de su turno	SI	SI

Implementación en el backend: Un segundo *middleware* de Express (`verifyRole('Admin')`) se encadena después de `verifyToken` en las rutas sensibles. Si el `req.user.role` no coincide con el rol requerido, el *middleware* interrumpe la cadena y retorna una respuesta 403 Forbidden con un mensaje de error descriptivo, sin ejecutar el controlador.

Implementación en el frontend: React renderiza condicionalmente los componentes de navegación y las opciones del menú lateral basándose en el rol almacenado en el estado de Redux. Un cajero nunca verá los botones de administración de productos ni el acceso al panel de reportes gerenciales, reduciendo la superficie de error operativo y mejorando la experiencia de usuario.

3.6.3 Módulo de Transacciones

Núcleo central del sistema TPS para la cafetería, diseñado en React.js para ofrecer una interfaz táctil de alta velocidad y baja fricción en la toma de pedidos (Punto de Venta).

- **Toma de Pedidos (POS):** Interfaz interactiva para seleccionar categorías (Cafés, Infusiones, Postres, Snacks) y agregar productos al carrito de compras con sus respectivas cantidades.
- **Gestión de Mesas y Estados:** Vinculación obligatoria de cada orden a una mesa específica de la cafetería. Control del flujo del pedido cambiando su estado: “En preparación” (barra/cocina), “Servido” y “Pagado”.
- **Procesamiento de Pago y Cierre:** Cálculo automático en tiempo real de subtotales, impuestos y total a cobrar. Registro del método de pago (efectivo, tarjeta) e impresión del comprobante o ticket de venta.
- **Historial de transacciones:** Bitácora inmutable en MongoDB de todas las ventas realizadas, asociadas al cajero en turno, con protección contra alteraciones concurrentes.

3.6.4 Módulo de Reportes

Módulo analítico estadístico que destila la información transaccional operativa de la cafetería para facilitar la toma de decisiones de la gerencia.

- **Dashboard Estadístico:** Panel visual en el *frontend* que emplea librerías de gráficos (ej. Chart.js o Recharts) para mostrar las métricas clave en tiempo real.

- **Reportes de Ventas:** Consultas agregadas a MongoDB para extraer ingresos diarios, semanales o mensuales.
- **Rendimiento de Productos:** Identificación automática de los productos más vendidos (ej. Capuchino, Croissants) y los de menor rotación en el menú.
- **Exportación de Datos:** Capacidad para generar y descargar los reportes consolidados por cajero o por turnos en formatos limpios como PDF o Excel, facilitando el arqueo de caja y la contabilidad externa.

3.7 Capa Backend Funcional

El *backend* de la cafetería está desarrollado en **Node.js** con el *framework* **Express.js**, actuando como una API RESTful robusta, aislada de la interfaz gráfica y conectada a **MongoDB**. Su arquitectura sigue el patrón MVC (Modelo-Vista-Controlador) adaptado a servicios:

- **Conexión BD:** Implementación de la cadena de conexión segura hacia MongoDB utilizando la librería mongoose para el modelado de datos mediante esquemas estructurados.
- **Modelos (Models):** Representación orientada a documentos de las entidades principales de la cafetería: UserSchema (personal operativo y administrativo), ProductSchema (menú), TableSchema (mesas) y OrderSchema (transacciones/ventas).
- **Controladores (Controllers):** Funciones que capturan las peticiones HTTP (GET, POST, PUT, DELETE), procesan la lógica central de ventas y devuelven respuestas estandarizadas en formato JSON.
- **Rutas y Servicios (Routes/Services):** Definición ordenada de los *endpoints* de la API (/api/orders, /api/products, etc.) extrayendo la lógica de negocio a un nivel de servicio para mantener controladores limpios.
- **Capa de Seguridad (Middlewares):** Bloques intermedios que protegen rigurosamente las rutas. Incluyen la verificación de autenticidad mediante la validación y descryptación de JSON Web Tokens (JWT) y la autorización por niveles (Ej.

bloqueando a un Cajero de la ruta de borrado de productos).

- **Validaciones:** Uso de librerías en el *backend* para verificar la integridad de los *payloads* antes de interactuar con la base de datos (ej. asegurar que una orden recibida contenga obligatoriamente el ID de una mesa válida y al menos un producto).

3.8 Validación y pruebas del sistema

El sistema asegura la calidad del producto final a través de distintas evaluaciones de estrés y rendimiento:

- **Pruebas unitarias:** Validando que las funciones matemáticas y servicios individuales del *backend* operen según la lógica.
- **Pruebas funcionales:** Ejecución de casos de uso (Ej: Qué sucede si el usuario ingresa un formato de fecha erróneo).
- **Pruebas de integración:** Ensayos del flujo Cliente hacia la API y hacia la base de datos extremo a extremo.
- **Pruebas de aceptación:** Pruebas finales realizadas con un entorno cercano a la organización para validación definitiva del *Product Owner*.

3.9 Desarrollo del prototipo funcional

A lo largo de los *sprints* se generan prototipos incrementales. En esta etapa el proyecto expone sus interfaces plenamente interactivas reflejando casos de éxito desde el inicio de sesión (*login*), hasta el registro exitoso de la operación. (*Incluir evidencias y capturas de pantalla reales aquí*).

3.10 Documentación de Ingeniería Completa

Se acompañan como anexos técnicos o repositorios vinculados:

- **Documentación funcional:** Incluye la Especificación de Requerimientos de Software (SRS), relevamiento documentado explícito e historias de usuario extendi-

das.

- **Documentación técnica:** El diagrama de la arquitectura desplegada, diccionarios de datos, modelo E/R completo y especificación paramétrica de API.
- **Documentación del sistema:** Manual de usuario para operadores, el manual técnico, directrices de instalación en entorno de servidor y parametrización de variables de entorno.
- **3.10 Documentación del código:** Detalla la estructura de directorios del proyecto MERN y las dependencias utilizadas en el sistema de la cafetería.
 - **Estructura del Proyecto:** Separación física y lógica entre el cliente (aplicación React en el directorio /frontend) y el servidor (API Node.js en el directorio /backend).
 - **Librerías y Dependencias Backend:** Uso de express para el enrutamiento HTTP, mongoose como ODM para modelar los datos de MongoDB, jsonwebtoken para la generación y firma de tokens de sesión, bcryptjs para el hash de contraseñas, y cors para habilitar peticiones seguras desde el frontend.
 - **Librerías y Dependencias Frontend:** Uso de react para la construcción de interfaces de usuario interactivas del POS, react-router-dom para la navegación entre el terminal de cobro y el panel de administración, y axios para consumir las rutas RESTful del backend asíncronamente.

Referencias Bibliográficas

- Brown, S. (2018). *Software architecture for developers: Visualising, documenting and exploring your software architecture*. Leanpub.
- Elmasri, R., & Navathe, S. B. (2015). *Fundamentos de sistemas de bases de datos* (7th ed.). Pearson Educación.
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON web token (JWT)* (RFC No. 7519). Internet Requests for Comments; RFC Editor. <https://doi.org/10.17487/RFC7519>
- Laudon, K. C., & Laudon, J. P. (2020). *Sistemas de información gerencial: Administración de la empresa digital* (16th ed.). Pearson Educación.
- O'Brien, J. A., & Marakas, G. M. (2011). *Sistemas de información gerencial* (10th ed.). McGraw Hill.
- Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. *Proceedings of the 1999 USENIX Annual Technical Conference*, 81–92.
- Schwaber, K., & Sutherland, J. (2020). *La guía definitiva de scrum: Las reglas del juego*. Scrum.org.
- Sommerville, I. (2015). *Ingeniería de software* (10th ed.). Pearson Educación.
- Stallings, W. (2017). *Fundamentos de seguridad en redes: Aplicaciones y estándares* (6th ed.). Pearson Educación.